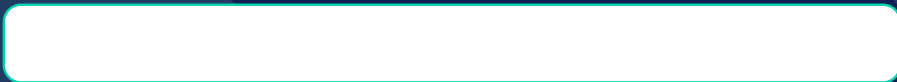




Testing Agentique

d'un Agent Conversationnel

Web & Mobile — piloté par OpenAI Swarm + Ollama



Plan

- 1 Pourquoi le Testing Agentique ?
- 2 OpenAI Swarm — Fondamentaux
- 3 Architecture du Système de Testing
- 4 Tools vs Code — L'approche moderne
- 5 Autonomous App Testing
- 6 Implémentation Swarm
- 7 Métriques & Résultats
- 8 Bonnes Pratiques & Prompt Engineering
- 9 Planning & Livrables
- 10 Conclusion

Le problème du testing conversationnel

Tester un agent, c'est difficile

Un agent conversationnel en production peut :

- **Halluciner** des faits inventés
- **Répondre différemment** sur Web vs Mobile
- Se **dégrader silencieusement** après une MAJ
- **Casser** sur des inputs inattendus ou vides
- Passer **des milliers de cas** que nul humain ne lit

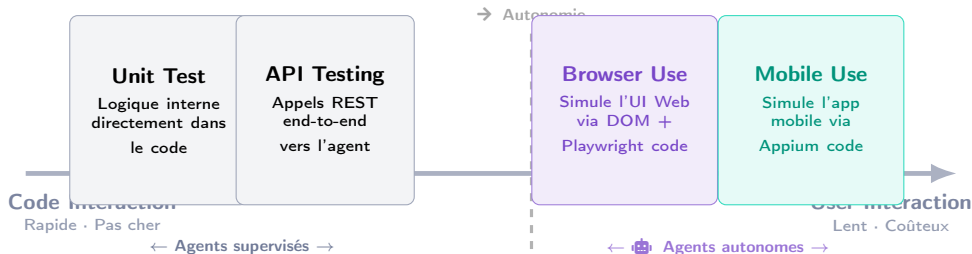
Le testing manuel : **lent, subjectif, non scalable.**

La solution : des agents qui testent

Autonomous App Testing = utiliser des agents IA pour :

- ✓ **Générer** les cas de test
- ✓ **Exécuter** sur Web et Mobile
- ✓ **Évaluer** la qualité des réponses
- ✓ **Détecter** les régressions auto.
- ✓ **Rapporter** sans intervention humaine

Autonomous App Testing — le spectre complet



Ce projet couvre l'ensemble : API (supervisé) + Browser Use + Mobile Use (autonome) via Swarm Agents.

OpenAI Swarm — Pourquoi ce framework ?

Swarm est un framework **minimaliste** d'orchestration multi-agents construit sur deux primitives :

- **Agent** = un SLM + des instructions système + des fonctions-outils
- **Handoff** = un agent passe la main à un autre agent selon le contexte

Pas de graphes complexes, pas d'état global caché.

Un agent = une responsabilité.

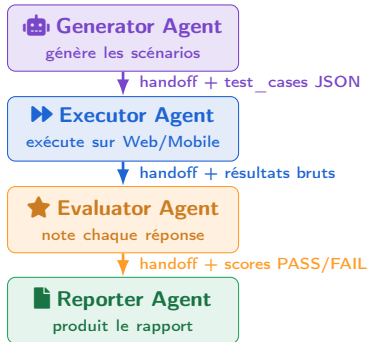
Pourquoi pas LangGraph ?

Critère	Swarm	LangGraph
Courbe appren.	Faible	Modérée
Verboité code	Faible	Élevée
Ollama compat.	Oui	Oui
Ideal pour	Proto rapide	Prod. complexe

➔ **Swarm = parfait pour apprendre le concept avant de migrer vers LangGraph.**

Le modèle Handoff — comment les agents coopèrent

L'état complet est passé dans le contexte à chaque handoff.

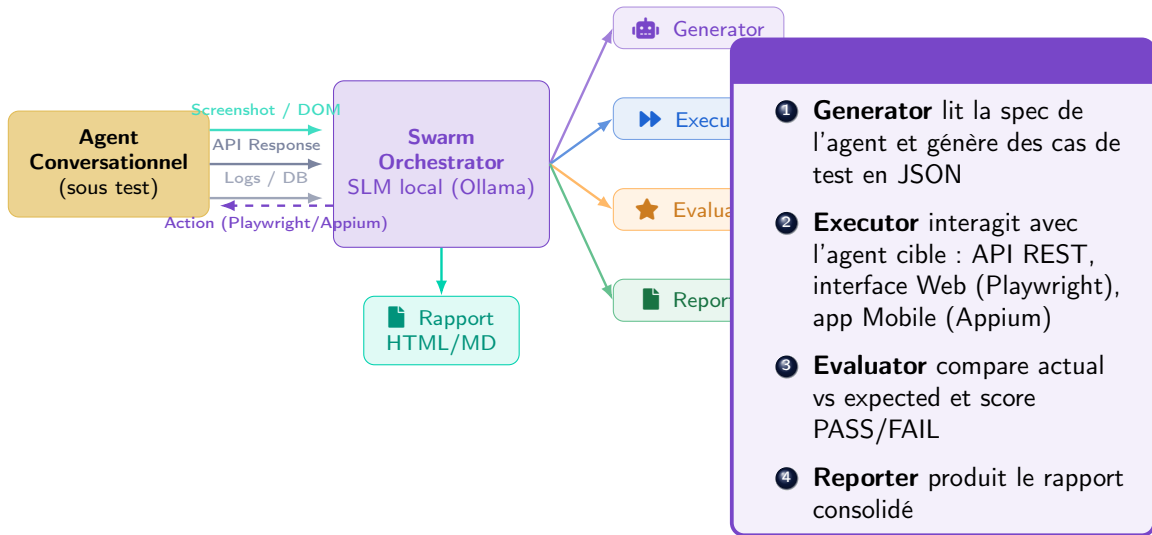


Un agent **ne fait qu'une seule chose** :

- **Generator** : lit une spec, retourne du JSON
- **Executor** : prend du JSON, interagit avec l'app
- **Evaluator** : prend résultats, retourne des scores
- **Reporter** : prend des scores, écrit un rapport

Chaque agent peut utiliser un **SLM différent** selon la complexité de sa tâche.

Architecture globale — Vue système



Les 3 canaux d'exécution

🔌 Canal 1 — API REST

Cible : endpoint /chat de l'agent

- Messages via httpx
- Validation schéma JSON
- Test latence, timeout, 5xx
- Inputs malformés, vides, XSS

Outil : httpx + pytest

Vitesse : très rapide

Cible : interface chat web (React/Vue)

- Navigation DOM complète
- Screenshot après chaque action
- Détection crash, spinner infini
- Flux login → chat → logout

Outil : Playwright (code-

📱 Canal 3 — Mobile

Cible : app mobile Android/iOS

- Gestes : tap, swipe, scroll
- Capture écran post-action
- Émulateur Android (AVD)
- Cohérence Web ↔ Mobile

Outil : Appium (code-based) / 23

Tools vs Code — Le débat fondamental

L'agent appelle des outils atomiques prédéfinis :

```
click(selector)
fill(selector, value)
drag(from, to)
new_tab(url)
```

Problèmes réels :

- Impossible d'énumérer *toutes* les actions possibles
- Chaque action = 1 appel LLM = latence cumulée élevée
- Fragile sur les interfaces dynamiques (React, SPA)

Approche Code-Based (adoptée)

L'agent **génère directement du code Playwright** :

```
page.goto("/chat")
page.fill("#input", message)
page.click("#send")
page.wait_for_selector(".reply")
page.screenshot(path="s1.png")
```

Avantages réels :

- Couverture **illimitée** — tout ce que Playwright sait faire
- **1 seul appel LLM** pour un scénario entier
- Rejouable, versionnable, diffable sur

Comparaison visuelle — même scénario, deux approches

```
1 # Appel 1 - LLM decide
2 tool_call("navigate", {"url": "/chat"})
3
4 # Appel 2 - LLM decide
5 tool_call("find_element", {"id": "input"})
6
7 # Appel 3 - LLM decide
8 tool_call("type_text", {"text": "Bonjour"})
9
10 # Appel 4 - LLM decide
11 tool_call("click", {"id": "send-btn"})
12
13 # Appel 5 - LLM decide
14 tool_call("get_text", {"class": ".bot-msg"})
```

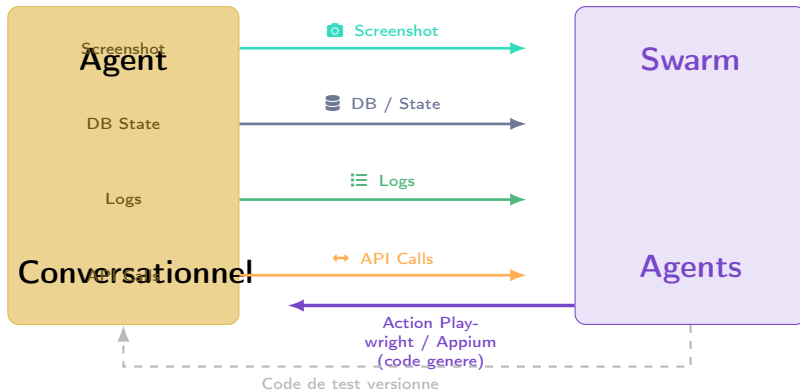
➔ 5 aller-retours LLM, 5x la latence.

Code-Based : 1 appel LLM

```
1 # 1 seul appel LLM, genere tout ca :
2 with sync_playwright() as p:
3     browser = p.chromium.launch()
4     page = browser.new_page()
5     page.goto("http://app/chat")
6     page.fill("#chat-input", "Bonjour")
7     page.click("#send-button")
8     page.wait_for_selector(
9         ".bot-message", timeout=10000)
10    reply = page.inner_text(
11        ".bot-message:last-child")
12    page.screenshot(path="test_01.png")
13    browser.close()
```

➔ 1 appel LLM, execution directe.

Autonomous App Testing — Architecture complète



L'agent intègre **4 sources simultanées** pour décider — pas seulement la réponse textuelle. C'est ce qui le rend **autonome**.

Autonomous App Testing — les 3 stratégies



Computer Use Testing

L'agent prend des **screenshots** et décide de la prochaine action visuellement.

- Très expressif : 1 :1 avec l'interface réelle
- Fonctionne sur **toute** app (même sans DOM)
- Nécessite un LLM vision
- **Lent et cher** — fallback uniquement

L'agent **génère du code Playwright** qui interagit avec le DOM.

- Rapide et précis
- Rejouable, diffable, CI-friendly
- Nécessite DOM accessible
- **Stratégie principale** de ce projet



API Testing

L'agent **génère des appels REST** directs vers l'agent conversationnel.

- Le plus rapide (ms)
- Pas d'UI nécessaire
- Limité à la logique API
- **Stratégie de régression** rapide

Usage : smoke tests, CI pre-merge

Generator Agent — Créer les cas de test

Generator Agent avec Ollama

```
1 from swarm import Swarm, Agent
2 from openai import OpenAI
3
4 # Swarm connecte a Ollama (interface OpenAI-compatible)
5 client = Swarm(client=OpenAI(base_url="http://localhost:11434/v1", api_key="ollama"))
6
7 def handoff_to_executor():
8     """Passe la main à l'Executor quand les cas de test sont prêts."""
9     return executor_agent
10
11 generator_agent = Agent(
12     name="Generator",
13     model="mistral",
14     instructions="""Tu es un expert QA spécialisé en agents conversationnels.
15 À partir d'une spécification, génère des cas de test exhaustifs.
16 Chaque cas doit contenir: id, canal (api/web/mobile), input utilisateur,
17 contexte de conversation, résultat attendu, critères d'échec.
18 Couvre: cas nominaux, cas limites, inputs vides, questions hors-domaine.
19 Retourne UNIQUEMENT un JSON valide avec la clé 'test_cases' (liste).
20 Quand tu as fini de générer, appelle handoff_to_executor.""",
21     functions=[handoff_to_executor],
22 )
23
24 # Lancement
25 response = client.run(
26     agent=generator_agent,
```

Executor Agent — Playwright code-based

⌘ Executor Agent — code Playwright genere par le SLM

```
1 import httpx, json
2 from playwright.sync_api import sync_playwright
3
4 def execute_api_test(test_case_json: str) -> str:
5     tc = json.loads(test_case_json)
6     r = httpx.post("http://localhost:8000/chat",
7                   json={"message": tc["input"], "session_id": "swarm_test"},
8                   timeout=30)
9     return json.dumps({"id": tc["id"], "canal": "api",
10                       "input": tc["input"], "expected": tc["expected"],
11                       "actual": r.json().get("response", ""), "status": r.status_code})
12
13 def execute_web_test(test_case_json: str) -> str:
14     tc = json.loads(test_case_json)
15     with sync_playwright() as p:
16         page = p.chromium.launch().new_page()
17         page.goto("http://localhost:3000/chat")
18         page.fill("#chat-input", tc["input"])      # Selector genere par le SLM
19         page.click("#send-button")
20         page.wait_for_selector(".bot-message", timeout=12000)
21         actual = page.inner_text(".bot-message:last-child")
22         page.screenshot(path=f"screenshots/{tc['id']}.png")
23         page.context.browser.close()
24     return json.dumps({"id": tc["id"], "canal": "web",
25                       "input": tc["input"], "expected": tc["expected"],
26                       "actual": actual, "screenshot": f"{tc['id']}.png"})
```

Evaluator + Reporter — Scoring et rapport

</> Evaluator et Reporter Agents

```
1 def handoff_to_reporter(): return reporter_agent
2
3 evaluator_agent = Agent(
4     name="Evaluator", model="llama3.2",
5     instructions="""Expert_QA_senior. Pour_chaque_resultat_de_test, evalue_selon:
6 1. Pertinence (1-5): la_reponse_repond-elle_bien_a_la_question?
7 2. Exactitude (1-5): les_faits_sont-ils_corrects?
8 3. Hallucination: le_modele_a-t-il_invente_des_informations? (true/false)
9 4. Coherence (1-5): coherent_avec_le_contexte_de_la_conversation?
10 Retourne_UNIQUEMENT_JSON: {id, scores, hallucination, verdict: PASS|FAIL, reason}
11 FAIL_immediat_si: hallucination=true OU score_moyen < 3.0 OU status_code >= 400.
12 Quand_tous_les_tests_sont_evalues, appelle_handoff_to_reporter.""",
13     functions=[handoff_to_reporter])
14
15 def save_markdown_report(content: str) -> str:
16     import pathlib
17     pathlib.Path("reports").mkdir(exist_ok=True)
18     pathlib.Path("reports/test_report.md").write_text(content)
19     return "Rapport_sauvegarde_dans_reports/test_report.md"
20
21 reporter_agent = Agent(
22     name="Reporter", model="mistral",
23     instructions="""Genere_un_rapport_Markdown_structure:
24 # Resume: taux_reussite_global, nombre_PASS/FAIL
25 # Tableau: id | canal | verdict | score | raison
26 # Top 3 echecs_critiques_avec_cause_probable
```

Testing Mobile — Appium code-based

⌕ Executor Mobile — code Appium genere par le SLM

```
1 from appium import webdriver
2 from appium.options import UiAutomator2Options
3
4 def execute_mobile_test(test_case_json: str) -> str:
5     tc = json.loads(test_case_json)
6
7     # Configuration emulateur Android
8     options = UiAutomator2Options()
9     options.platform_name = "Android"
10    options.device_name = "emulator-5554"
11    options.app_package = "com.monapp.chatbot"
12    options.app_activity = ".ChatActivity"
13
14    driver = webdriver.Remote("http://localhost:4723", options=options)
15
16    try:
17        # Code genere par le SLM      meme principe que Playwright
18        chat_input = driver.find_element("id", "com.monapp.chatbot:id/chat_input")
19        chat_input.send_keys(tc["input"])
20
21        send_btn = driver.find_element("id", "com.monapp.chatbot:id/send_btn")
22        send_btn.click()
23
24        from appium.webdriver.common.appiumby import AppiumBy
25        driver.implicitly_wait(10)
26        reply_el = driver.find_element(AppiumBy.XPATH, "//android.widget.TextView[@resource-id='bot_reply"]
```


Métriques — Ce qu'on mesure et pourquoi

⚡ Métriques automatiques

Performance :

- Temps de réponse (ms) par canal
- Taux de timeout ($> 30s$) — FAIL
- Disponibilité de l'agent (%)

Qualité NLP :

- BLEU / ROUGE vs réponse de référence
- Taux de refus inappropriés
- Cohérence JSON (parse ok ?)
- Détection hallucination (NLI score)

Scoring simulé (1–5) :

- Pertinence de la réponse à l'intent
- Exactitude factuelle vérifiable
- Cohérence contexte conversation
- Format de sortie respecté

Flags FAIL bloquants :

- Hallucination détectée = **FAIL immédiat**
- Crash UI / erreur 5xx = **FAIL immédiat**
- Divergence Web $\leftrightarrow$ Mobile $> 20\%$ = 17 / 23

Exemple de rapport de test

📊 Tableau résultats — exemple attendu

Scénario	Canal	Latence	Score	Halluc.	Verdict
Salutation simple	API	0.7s	4.6	Non	PASS
Question produit	Web	1.4s	4.1	Non	PASS
Suivi commande #1234	Mobile	2.8s	3.9	Non	PASS
Input vide	API	0.2s	1.0	Non	FAIL
Chiffre inventé	Web	3.1s	2.5	Oui	FAIL
Question hors-domaine	Mobile	4.2s	3.5	Non	PASS
Session expirée	Web	0.4s	1.2	Non	FAIL
Conversation longue	API	5.1s	3.8	Non	PASS
Taux de réussite global :				62.5% (5/8)	

⚠ Hallucination sur chiffres = FAIL bloquant #1. Input vide non géré = FAIL UX critique #2.

Prompt Engineering pour agents de test

"Teste si l'agent répond bien à cette question."

Problèmes : pas de critères, pas de format, pas de seuil de FAIL, réponse subjective, impossible à parser automatiquement.

Prompt structuré pour Evaluator

Tu es QA senior. Evalue ce test :

INPUT: {input}

EXPECTED: {expected}

ACTUAL: {actual}

CANAL: {canal}

Retourne UNIQUEMENT ce JSON :

```
{  
  "pertinence": 1-5,  
  "exactitude": 1-5,  
  "hallucination": true/false,  
  "verdict": "PASS" ou "FAIL",  
  "reason": "explication courte"  
}
```

FAIL si hallucination=true ou score<3.

★ Les 4 règles d'or pour les agents de test

1. Rôle précis ("QA senior")
2. Format de sortie strict (JSON uniquement)
3. Toujours expected

Planning 7 semaines

S1–S2 : Fondations

-  Lire : Swarm, Playwright, Appium, Ollama
-  Setup : Ollama + 3 SLMs locaux
-  Tester l'agent cible manuellement
-  Écrire 15 cas de test manuels en JSON

S3–S4 : Pipeline Swarm

-  Generator + Executor API
-  Evaluator + Reporter
-  Pipeline end-to-end sur 10 cas

S5–S6 : Web & Mobile

-  Intégrer Playwright (Browser Use)
-  Intégrer Appium (émulateur Android)
-  Screenshots + analyse visuelle
-  Comparaison Web vs Mobile auto.

S7 : Finalisation

-  Rapport final (50+ scénarios)
-  Dashboard HTML (Plotly)
-  Démo live du système
-  Piste CI/CD : tests pre-merge

Code à produire

① Pipeline Swarm complet

- Generator / Executor / Evaluator / Reporter
- Canal API + Web (Playwright) + Mobile (Appium)
- Export JSON + Markdown

② Corpus de test

- 50+ scénarios Web & Mobile
- Cas nominaux + limites + adversariaux
- Versionné en JSON

Documents à produire

① README : setup, archi, usage

② Rapport de testing :

- Tableau résultats par scénario
- Analyse échecs critiques
- Divergences Web vs Mobile
- Recommandations

③ Présentation + démo live

 **Critère de succès** : le système détecte **automatiquement** les régressions entre deux versions de l'agent cible.

Ce que vous allez acquérir



Compétences techniques

- **Agentic Testing** — testing autonome piloté par LLM
- **OpenAI Swarm** — orchestration multi-agents
- **Playwright code-based** — browser automation moderne
- **Appium** — mobile automation Android/iOS
- **Prompt Engineering** orienté QA strict
- **SLMs locaux** (Ollama) pour pipelines privés



Valeur ajoutée réelle

- Concevoir un système **autonome de bout en bout**
- Comprendre **pourquoi code-based bat tool-based**
- Maîtriser la **détection d'hallucination** en prod
- Penser **QA Engineering + AI Engineering** simultanément
- Savoir **évaluer objectivement** un agent sans biais humain

Ce projet vous place à la frontière entre QA Engineering et AI Engineering — compétence rare, très demandée en 2025.



C'est à vous de jouer !

Construisez le testing que vous auriez voulu avoir

Département IA — 18 juin 2026